



CSCI235 – Database Systems

Functional Dependencies

sjapit@uow.edu.au

29 March 2026



Acknowledgements

The following presentation were adapted from the lecture slides of:

CSCI235 – Database Systems, 02 Functional Dependencies

By Dr Janusz R. Getta, University of Wollongong, Australia

Outline

- Functional dependency? What is it?
- Functional dependencies versus classes of objects
- Functional dependencies versus associations
- Derivations of functional dependencies
- Armstrong axioms
- Other inference rules
- Using inference rules

Functional dependency? What is it?

Recap:

- Functional dependency (FD) is **a set of constraints between two set of attributes in a relation.**
- Given a relational table $R(A_1, A_2, A_3, \dots, A_n)$, and a functional dependency $X \rightarrow Y$ is valid in a relational schema R , functional dependency says that if two tuples have same values for attributes $a_1, a_2, \dots, a_m$ (X), then those two tuples must have same values for attributes $b_1, b_2, \dots, b_p$ (Y). it is not possible that R has two rows that agree in the components for all attributes in a set X yet disagree on one or more component for the attributes in a set Y .

Functional dependency? What is it?

Enrollments Table

StudentID	CourseCode	Semester	Grade
101	CS101	Fall2023	A
101	MATH101	Fall2023	B
102	CS101	Fall2023	B
101	CS101	Spring2024	A

Same studentID, CourseCode, different Semester and Grade

- A **functional dependency (FD)** describes a relationship between two sets of attributes in a table.

Functional dependency? What is it?

More Examples:

- A warehouse is located at exactly one address:
warehouse → address
- An address is related to exactly one warehouse:
address → warehouse
- At a warehouse, the parts of the same sort have only one total quantity:
warehouse, part → quantity
- A car has one owner:
registration → drivingLicense

Functional dependency? What is it?

More examples

- A student has one first name and one last name and one date of birth:

studentNumber

→ firstName, lastName, dateOfBirth

- An employee belongs to one department:

employeeNumber → departmentName

- A department has one manager:

departmentName → managerNumber

- An employee has one manager:

employeeNumber → managerNumber

Functional dependency? What is it?

More examples

- A student enrolls a subject one time:
 $studentNumber, subjectCode \rightarrow enrolmentDate$
- An employee is located in one building in one office:
 $employeeNumber \rightarrow buildingNumber, officeNumber$
- An office in a building hosts one employee:
 $buildingNumber, officeNumber \rightarrow employeeNumber$
- An office in a building at a campus hosts one employee:
 $campusName, buildingNumber, officeNumber \rightarrow employeeNumber$

Functional dependency? What is it?

More examples

- A department has one manager:
departmentName $\rightarrow$ *managerNumber*
- A department is located in one building:
departmentName $\rightarrow$ *buildingNumber*
- A department has one manager and it is located in one building:
departmentName
 $\rightarrow$ *managerNumber, buildingNumber*

Functional dependency? What is it?

How to discover the **functional dependencies** in a relational table?

- Is it possible to discover the **functional dependencies** in a **relational schema** (a header of relational table) $R(A, B, C, D, E)$?
- Of course it is impossible to do it because we do not know the **semantics (the meanings)** of the names: R, A, B, C, D, E
- To discover the **functional dependencies** in a relational table we **MUST** use the **semantics** of a **relational table name** and the **names of attributes**

Functional dependency? What is it?

- For example consider a relational schema (a header of relational table)
TRIP(*rego#*, *licence#*, *tdate*) of a relational table that contains information about the **trips** made by the **drivers** (*licence#*) who used the **trucks** (*rego#*) on a given **day** (*tdate*)
- Can a truck be used only one time ?
If yes then *rego#* $\rightarrow$ *tdate*
- Can a driver make only one trip ?
If yes the *licence#* $\rightarrow$ *tdate*

Functional dependency? What is it?

- Can a driver use more than one truck ?
If yes then *licence#* $\rightarrow$ *rego#*
- Can a truck be used by more than one driver ?
If yes then *rego#* $\rightarrow$ *licence#*
- And so on ...

How to derive/determine functional dependency?

Practical Algorithm

- Here's a systematic approach you can always follow:
 - **List all single columns** - Check each for uniqueness
 - **For non-unique single columns** - Try combinations starting with 2 columns
 - **Use the "removal test"** - If you can remove any column and still have uniqueness, it's not minimal
 - **Stop when you find the smallest unique combination** - There might be multiple minimal superkeys!

Functional dependencies versus classes of objects

A class of object **STUDENT**.

STUDENT	
stdNum	ID1
stdFName	ID2
stdLName	ID2
stdDOB	ID2
average	
language	[1..*]

Validates (satisfies) the following functional dependencies:

stdNum $\rightarrow$ *stdFName*

stdNum $\rightarrow$ *stdLName*

stdNum $\rightarrow$ *stdDOB*

stdNum $\rightarrow$ *average*

stdFName, stdLName, stdDOB $\rightarrow$ *stdNum*

stdFName, stdLName, stdDOB $\rightarrow$ *average*

Functional dependencies versus classes of objects

stdNum $\rightarrow$ *stdFName*

stdNum $\rightarrow$ *stdLName*

stdNum $\rightarrow$ *stdDOB*

stdNum $\rightarrow$ *average*

The four functional dependencies shown above are equivalent to:

stdNum $\rightarrow$ *stdFName, stdLName, stdDOB, average*

Functional dependencies versus classes of objects

stdFName, stdLName, stdDOB $\rightarrow$ stdNum

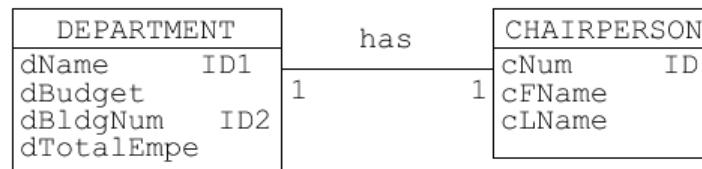
stdFName, stdLName, stdDOB $\rightarrow$ average

The two functional dependencies shown above are equivalent to:

stdFName, stdLName, stdDOB $\rightarrow$ stdNum, average

Functional dependencies versus associations

The classes of objects **DEPARTMENT** and **CHAIRPERSON** and association **has**



validate (satisfy) the following functional dependencies:

$dName \rightarrow dBudget, dBldgNum, dTotalEmpe$

$dBldgNum \rightarrow dName, dBudget, dTotalEmpe$

$cNum \rightarrow cFName, cLName$

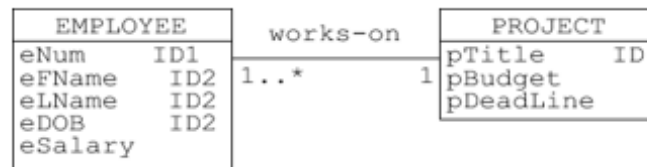
$dName \rightarrow cNum, cFName, cLName$

$dBldgNum \rightarrow cNum, cFName, cLName$

$cNum \rightarrow dName, dBudget, dBldgNum, dTotalEmpe$

Functional dependencies versus associations

The classes of objects **EMPLOYEE** and **PROJECT** and association **works-on**



validate (satisfy) the following functional dependencies:

$eNum \rightarrow eFName, eLName, eDOB, eSalary$

$eFName, eLName, eDOB \rightarrow eNum, eSalary$

$pTitle \rightarrow pBudget, pDeadLine$

$eNum \rightarrow pTitle, pBudget, pDeadLine$

$eFName, eLName, eDOB \rightarrow pTitle, pBudget, pDeadLine$

Functional dependencies versus associations

The classes of objects **STUDENT** and **COURSE** and association **enroll**

validate (satisfy) the following functional dependencies:

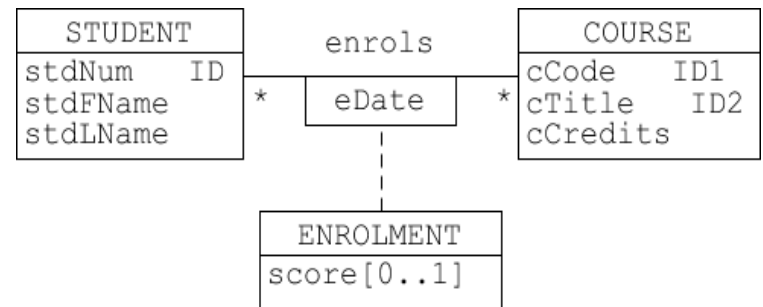
stdNum $\rightarrow$ *stdFName*, *stdLName*

cCode $\rightarrow$ *cTitle*, *cCredits*

cTitle $\rightarrow$ *cCode*, *cCredits*

stdNum, *cCode*, *eDate* $\rightarrow$ *score*

stdNum, *cTitle*, *eDate* $\rightarrow$ *score*



Derivations of functional dependencies

Consider a relational schema (a header of relational table)

EMPLOYEE(e#, ename, department, address, chairperson)

If $e\# \rightarrow ename$ and $e\# \rightarrow department$

Then $e\# \rightarrow ename, department$

If $e\# \rightarrow department$ and $department \rightarrow address$

Then $e\# \rightarrow address$

If $e\# \rightarrow department$ and $department \rightarrow chairperson$

Then $e\# \rightarrow chairperson$

Derivations of functional dependencies

If $e\# \rightarrow department$

then $e\#, ename \rightarrow department$

If $e\#, ename \rightarrow department$ then

$e\#, ename, address \rightarrow department$

Derivations of functional dependencies

It is always true that $e\# \rightarrow e\#$

Functional dependency $e\# \rightarrow e\#$ is called as a **trivial functional dependency**

It is always true that $e\#, ename \rightarrow e\#$

A functional dependency $e\#, ename \rightarrow e\#$ is also called as a **trivial functional dependency**

A **trivial functional dependency** is a functional dependency that is **always true** no matter what its left and right hand sides are.

Derivations of functional dependencies

Consider a relational schema $R(A, B, C)$

It is always true that $A \rightarrow A$

It is always true that $A, B \rightarrow A$

It is always true that $A, B, C \rightarrow A$

If $A \rightarrow B$ then $A, C \rightarrow B$

If $A \rightarrow B, C$ then $A \rightarrow B$ and $A \rightarrow C$

If $A \rightarrow B$ and $B \rightarrow C$ then $A \rightarrow C$

Armstrong axioms

Let $R = (A_1, \dots, A_n)$ be a relational schema (a header of relational table) and

let X, Y, Z be the nonempty subsets of $\{A_1, \dots, A_n\}$

1. If $Y \subseteq X$ then $X \rightarrow Y$ (**reflexivity axiom**)
2. If $X \rightarrow Y$ then $X, Z \rightarrow Y, Z$ (**augmentation axiom**)
3. If $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$ (**transitivity axiom**)

The axioms 1, 2, and 3 form a **minimal and complete set of axioms**

Other inference rules

Let $R = (A_1, \dots, A_n)$ be a relational schema (a header of relational table) and let X, Y, Z be the nonempty subsets of $\{A_1, \dots, A_n\}$

1. If $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow Y, Z$ (**union rule**)
2. If $A \rightarrow B$ and $X \rightarrow Y$ then $AX \rightarrow BY$ (**composition rule**)
3. If $X \rightarrow Y$ and $Z \subseteq Y$ then $X \rightarrow Z$ (**decomposition rule** or **reduce right hand side rule**)
4. If $X \rightarrow Y$ and $W, Y \rightarrow Z$ then $W, X \rightarrow Z$ (**pseudo transitivity rule**)
5. If $X \rightarrow Y$ then $X, Z \rightarrow Y$ (**extend left hand side rule**)

Using inference rules

Let $R = (A, B, C)$ be a relational schema

Given set of functional dependencies

$$F = \{A \rightarrow B, B \rightarrow C\} \text{ valid in } R$$

Is it true that $A \rightarrow C$?

If $A \rightarrow B$ and $B \rightarrow C$ then application of **transitivity axiom** provides $A \rightarrow C$

Using inference rules

Let $R = (A, B, C)$ be a relational schema

Given set of functional dependencies

$F = \{A \rightarrow B, C\}$ valid in R

Is it true that $A \rightarrow B$ and $A \rightarrow C$?

Reflexivity axiom provides $B, C \rightarrow C$

If $A \rightarrow B, C$ and $B, C \rightarrow C$ then **transitivity axiom** provides $A \rightarrow C$

Reflexivity axiom provides $B, C \rightarrow B$

If $A \rightarrow B, C$ and $B, C \rightarrow B$ then **transitivity axiom** provides $A \rightarrow B$

Using inference rules

Let $R = (A, B, C)$ be a relational schema

Given set of functional dependencies

$F = \{A \rightarrow B, A \rightarrow C\}$ valid in R

Is it true that $A \rightarrow B, C$?

If $A \rightarrow B$ then **augmentation axiom** provides
 $A \rightarrow A, B$

If $A \rightarrow C$ then **augmentation axiom** provides
 $A, B \rightarrow B, C$

If $A \rightarrow A, B$ and $A, B \rightarrow B, C$ then **transitivity axiom**
provides
 $A \rightarrow B, C$

Using inference rules

Let $R = (A, B, C)$ be a relational schema

Given set of functional dependencies $F = \{A \rightarrow B\}$
valid in R

Is it true that $A, C \rightarrow B$?

Reflexivity axiom provides $A, C \rightarrow A$

If $A, C \rightarrow A$ and $A \rightarrow B$ then **transitivity axiom**
provides $A, C \rightarrow B$

Using inference rules

A relational schema

STUDENT(***s#***, ***fname***, ***lname***, ***dob***, ***average***) validates (satisfies) the following functional dependencies:

s# $\rightarrow$ ***fname***

s# $\rightarrow$ ***lname***

s# $\rightarrow$ ***dob***

s# $\rightarrow$ ***average***

fname, lname, dob $\rightarrow$ ***s#***

fname, lname, dob $\rightarrow$ ***average***

Using inference rules

Hence,

$s\# \rightarrow fname, lname, dob, average$ and ...
 $fname, lname, dob \rightarrow s\#, average$

Note, that both functional dependencies **cover** entire relational schema and **no other** functional dependencies that **do not cover** entire relational schema validate in the schema e.g.

$fname \rightarrow s\#$ or *$lname \rightarrow s\#$* or *$dob \rightarrow s\#$* etc.

The relational schema

STUDENT(*$s\#, fname, lname, dob, average$*) is now normalised.

References

T. Connolly, C. Begg, Database Systems, A Practical Approach to Design, Implementation, and Management, Chapter 14.4 Functional Dependencies, Chapter 15.1 More on Functional Dependencies, Pearson Education Ltd, 2015